



*bo programowanie powinno mieć flow a nie chaos*

*framework wspierający rozwój oprogramowania kierowany specyfikacją*

Błażej Strus

maj 2026

## Streszczenie

Niniejsze opracowanie opisuje blast, framework wspomagający rozwój oprogramowania we współpracy z modelami językowymi, zbudowany w środowisku Claude Code. Narzędzie porządkuje cały cykl tworzenia funkcjonalności jako deterministyczną sekwencję faz: od specyfikacji wymagań, przez projekt techniczny i rozpisanie zadań, do implementacji, audytu bezpieczeństwa oraz synchronizacji pamięci projektu. Każde przejście pomiędzy fazami zabezpieczone jest bramką akceptacji egzekwowaną na poziomie środowiska uruchomieniowego, niezależnie od decyzji modelu.

Najbardziej charakterystycznym elementem systemu jest panel oceniający uruchamiany dla decyzji wysokiego ryzyka. Ten sam artefakt analizowany jest równolegle przez modele należące do trzech różnych dostawców: Anthropic (rodziny Sonnet i Opus), Google (model Gemini) oraz lokalny model Qwen pracujący przez bibliotekę Ollama na sprzęcie posiadanym przez użytkownika. Wyniki scala czwarty komponent, agregator oparty na modelu Haiku. W odróżnieniu od typowych rozwiązań wielomodelowych, agregator oznacza jednomyślną zgodę bez głosu odrębnego jako sytuację wymagającą weryfikacji. Założenie wynika z opublikowanego w 2026 roku benchmarku M3MAD-Bench, według którego jednomyślność w realnych warunkach roboczych jest statystycznie nietypowa i często sygnalizuje kaskadę pewności siebie, a nie potwierdzenie poprawności.

Porównanie z czterema rozwiązaniami referencyjnymi dostępnymi w 2026 roku (Kiro firmy Amazon, GitHub Spec Kit, BMAD-METHOD oraz mniejsze projekty) wskazuje, że blast plasuje się powyżej średniej w dziewięciu z dwudziestu jeden ocenianych wymiarów technicznych, na poziomie porównywalnym w ośmiu, poniżej w czterech. Do obszarów przewagi należą: wielodostawcze panele oceniające, kierowanie zapytań do najtańszego modelu zdolnego dostarczyć wymaganą jakość, tryb pełnej prywatności blokujący komunikację zewnętrzną, rejestr komponentów wspólny dla całego repozytorium oraz automatyczna pętla samodoskonalenia. Obszary niedoboru dotyczą głównie dystrybucji i integracji z różnymi środowiskami programistycznymi.

# Spis treści

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Streszczenie .....                                              | 2  |
| 1. Wprowadzenie .....                                           | 5  |
| 1.1. Problem.....                                               | 5  |
| 1.2. Odpowiedź dyscyplinowana .....                             | 5  |
| 1.3. Cel niniejszego opracowania .....                          | 5  |
| 2. Stan rozwiązań w 2026 roku.....                              | 6  |
| 2.1. Kiro firmy Amazon .....                                    | 6  |
| 2.2. GitHub Spec Kit.....                                       | 6  |
| 2.3. BMAD-METHOD.....                                           | 7  |
| 2.4. Inne wymienne projekty .....                               | 7  |
| 2.5. Wzorce wspólne i obszary nieobsłużone .....                | 8  |
| 3. Architektura blast .....                                     | 8  |
| 3.1. Układ repozytorium.....                                    | 8  |
| 3.2. Konstytucja i warstwa kierująca .....                      | 9  |
| 3.3. Maszyna stanów pojedynczej specyfikacji .....              | 9  |
| 3.4. Granice determinizmu .....                                 | 10 |
| 4. Sekwencja faz pracy.....                                     | 10 |
| 4.1. Graf przejść .....                                         | 10 |
| 4.2. Bramki akceptacji.....                                     | 12 |
| 4.3. Wykaz komend.....                                          | 12 |
| 4.4. Fazy opcjonalne.....                                       | 13 |
| 5. System person .....                                          | 13 |
| 5.1. Uzasadnienie .....                                         | 13 |
| 5.2. Obsada .....                                               | 14 |
| 5.3. Mechanizm wykrywania własnej stronniczości w praktyce..... | 15 |
| 5.4. Persona a agent.....                                       | 16 |
| 6. Walidacja wielomodelowa.....                                 | 16 |
| 6.1. Uzasadnienie podejścia.....                                | 16 |
| 6.2. Kompozycje paneli.....                                     | 17 |
| 6.3. Mechanika protokołu B .....                                | 17 |
| 6.4. Zabezpieczenie przed kaskadą konsensusu .....              | 18 |

|                                                               |    |
|---------------------------------------------------------------|----|
| 6.5. Kierowanie zapytań do tańszych modeli .....              | 18 |
| 6.6. Tryb prywatności.....                                    | 19 |
| 7. Mechanizmy egzekwowania determinizmu .....                 | 20 |
| 7.1. Trzy mechanizmy kontrolne .....                          | 20 |
| 7.2. Uzasadnienie ogólne.....                                 | 20 |
| 7.3. Wzorzec orkiestratora.....                               | 21 |
| 8. Pamięć projektu i samodoskonalenie .....                   | 21 |
| 8.1. Warstwa kierująca jako pamięć.....                       | 21 |
| 8.2. Baza wiedzy i kuratorska SOTA.....                       | 22 |
| 8.3. Pętla samodoskonalenia.....                              | 22 |
| 8.4. Rejestr komponentów wspólny dla całego repozytorium..... | 23 |
| 9. Cykl życia poza wdrożeniem.....                            | 23 |
| 10. Ocena rozwiązania .....                                   | 24 |
| 10.1. Macierz porównawcza .....                               | 24 |
| 10.2. Dowody empiryczne.....                                  | 26 |
| 10.3. Pozycja blast w stosunku do stanu wiedzy .....          | 27 |
| 11. Ograniczenia i kierunki rozwoju.....                      | 27 |
| 11.1. Sprzężenie z jednym środowiskiem programistycznym.....  | 27 |
| 11.2. Dystrybucja i pakowanie.....                            | 28 |
| 11.3. Plateau debaty .....                                    | 28 |
| 11.4. Współbieżność w fazie implementacji .....               | 28 |
| 11.5. Ryzyko ról zastępczych .....                            | 28 |
| 11.6. Koszt walidacji dla małych funkcjonalności .....        | 29 |
| 12. Podsumowanie .....                                        | 29 |
| Literatura .....                                              | 30 |
| Dodatek A. Wykaz komend systemu.....                          | 32 |
| A.1. Fazy potoku.....                                         | 32 |
| A.2. Fazy walidacji.....                                      | 33 |
| A.3. Cykl życia .....                                         | 33 |
| A.4. Kompozycje i komendy meta.....                           | 33 |
| Dodatek B. Słownik .....                                      | 35 |

# 1. Wprowadzenie

## 1.1. Problem

Od połowy 2025 roku znacząca część zadań programistycznych delegowana jest do modeli językowych. Domyślny tryb porażki tej współpracy, określany w środowisku praktyków jako *vibe coding* (programowanie pod nastrój), przebiega podobnie: użytkownik formułuje życzenie, model produkuje kod, który wygląda wiarygodnie, a praca toczy się dalej bez wspólnej specyfikacji, bez testu, bez kroku weryfikacyjnego, bez kontraktu, który mogłaby przejąć następna osoba lub późniejsza wersja użytkownika. Artefaktem pracy staje się zapis rozmowy. Zapis rozmowy jest kruchy. Kod, który przetrwa, robi to zwykle przez przypadek.

Rozwój kierowany specyfikacją (ang. Spec-Driven Development, w skrócie SDD) jest ustrukturyzowaną odpowiedzią na ten problem. Założenie wyjściowe brzmi: skoro model produkuje najlepszy kod, gdy ma jasne wejście (wymagania, projekt, zadania), właściwą dyscypliną jest uczynienie tych wejść artefaktami pierwszej kategorii, weryfikowanymi przez człowieka zanim kod powstanie. Kod staje się wynikiem deterministycznego potoku faz, a nie założeniem. Potok podlega formalizacji w narzędziu, artefakty pozostają własnością człowieka.

## 1.2. Odpowiedź dyscyplinowana

W latach 2024 i 2026 powstało kilka frameworków przyjmujących to założenie. Kiro firmy Amazon zaoferowała środowisko programistyczne (IDE) oparte na widoku trzech artefaktów: wymagań, projektu oraz listy zadań. GitHub opublikował Spec Kit, framework w którym osobnym dokumentem jest konstytucja projektu (*constitution.md*) zawierająca dziewięć artykułów rządzących wszystkimi późniejszymi fazami. Społeczność programistów wytworzyła BMAD-METHOD, OpenSpec, Tessel, Agent OS oraz długi ogon mniejszych narzędzi. W pierwszym kwartale 2026 roku przeglądowy artykuł Martina Fowlera w wydawnictwie IEEE pod tytułem *Spec-Driven Development is Eating Software Engineering* zmapował ponad trzydzieści takich rozwiązań.

## 1.3. Cel niniejszego opracowania

blast jest jednym z tych frameworków. Motywacją do opracowania osobnego dokumentu opisującego jego architekturę jest fakt, że projekt skonwergował na zbiorze decyzji

indywidualnie nieszczególnych, ale wspólnie nietypowych. Dokument przedstawia: opis referencyjny architektury i potoku faz dokładny względem stanu repozytorium na dzień 7 maja 2026, macierz porównawczą z czterema frameworkami referencyjnymi w dwudziestu jeden wymiarach technicznych, opis systemu walidacji wielomodelowej z mechanizmem zabezpieczającym przed kaskadą konsensusu, oraz pełny wykaz komend i agentów (dodatek

## **2. Stan rozwiązań w 2026 roku**

Lista poniżej obejmuje cztery rozwiązania najczęściej referowane w 2026 roku oraz wzorce wspólne dla szerszej populacji.

### **2.1. Kiro firmy Amazon**

Kiro to środowisko programistyczne wywodzące się z otwartego kodu Visual Studio Code, opublikowane przez Amazon Web Services w lipcu 2025 roku. Dla każdej specyfikacji generuje trzy pliki: requirements.md (notacja EARS), design.md oraz tasks.md. Posiada system reguł uruchamianych przy zapisie pliku, na przykład: regeneruj testy gdy zmieni się ten plik źródłowy. Modelem językowym jest Claude Sonnet wywoływany przez usługę Bedrock. Cena dla użytkownika indywidualnego wynosi 20 dolarów miesięcznie.

Mocne strony rozwiązania to ścisła integracja z IDE, notacja EARS jako format domyślny utrzymujący wymagania w postaci możliwej do parsowania, oraz dojrzały system reguł plikowych. Ograniczenia: zależność od konkretnego środowiska programistycznego, brak modeli alternatywnych poza Sonnet, brak walidacji wielomodelowej, brak rejestru komponentów wspólnego dla wielu specyfikacji, konstytucja projektu jest ukryta w ustawieniach przestrzeni roboczej zamiast funkcjonować jako wersjonowany dokument.

### **2.2. GitHub Spec Kit**

GitHub opublikował Spec Kit pod koniec 2025 roku jako otwarty zestaw narzędzi do rozwoju kierowanego specyfikacją. Najistotniejszą cechą jest warstwa konstytucyjna, czyli plik constitution.md zawierający dziewięć artykułów rządzących późniejszymi fazami, oraz bramka kontrolna o numerze minus jeden porównująca każdą propozycję specyfikacji z zapisami konstytucji zanim projekt wejdzie w fazę zerową (specyfikacja). Sekwencja komend

ma postać: konstytucja, specyfikacja, plan, zadania, implementacja. Działa z trzema klientami: GitHub Copilot, Claude Code, Gemini CLI.

Do mocnych stron należy najbardziej rygorystyczna abstrakcja governance w polu (warstwa konstytucyjna jako artefakt z historią wersji), oraz dopracowane narzędzie do tworzenia nowych projektów (specify init). Ograniczenia: każda faza wykonywana jest pojedynczym agentem, brak walidacji wielomodelowej, brak systemu person, brak kierowania zapytań do tańszych modeli, brak trybu prywatności. Artykuły są rygorystyczne, ale nieliczne.

## **2.3. BMAD-METHOD**

BMAD-METHOD (Breakthrough Method for Agile AI-Driven Development) wypuścił wersję 6.6.0 w kwietniu 2026 roku i osiągnął 46,2 tysiąca gwiazdek na platformie GitHub. Wyróżniającym wkładem rozwiązania jest architektura skills, czyli kompozycyjne jednostki funkcji agenta przechowywane jako pliki YAML. Skills można stosować i odwoływać do nich w kolejnych fazach cyklu wytwarzania. BMAD posiada również dojrzały system person (nazwani agenci dla poszczególnych faz), w duchu zbliżony do rozwiązania zastosowanego w blast.

Mocne strony: największa społeczność, kompozycyjność skills (dodanie nowej funkcjonalności sprowadza się do napisania skill, a nie edycji komendy), dojrzały system person, wsparcie wielu silników agentowych. Ograniczenia: brak walidacji wielodostawczej (jeden agent na skill w danej chwili), brak mechanizmów na poziomie środowiska uruchomieniowego (dyscyplina skills opiera się na podpowiedziach kontekstowych), brak jawnej konstytucji, kierowanie zapytań do tańszych modeli nie jest centralne dla projektu.

## **2.4. Inne wymienne projekty**

OpenSpec to projekt minimalistyczny, oparty wyłącznie na języku Markdown, bez integracji agentowej. Targetuje czysto specyfikacyjne procesy z przekazaniem do dowolnego modelu. Tessl koncentruje się na generacji kierowanej testami. Agent OS to środowisko wieloagentowe nakierowane na długo działające procesy autonomiczne, z mniejszym naciskiem na dyscyplinę specyfikacji. Spec Kitty jest społecznościową wariacją Spec Kit z dodatkowymi szablonami.

## 2.5. Wzorce wspólne i obszary nieobsłużone

Wszystkie wymienione frameworki dzielą trzy wzorce: trzyfazowy potok artefaktów (wymagania, projekt, zadania, kod), Markdown jako wspólny język artefaktów, bramki akceptacji pomiędzy fazami. System person spotyka się w BMAD i blast, częściowo w pozostałych. Konstytucja projektu jako odrębny dokument występuje w Spec Kit i blast. Walidacja wielomodelowa, tryb prywatności, rejestr komponentów wspólny dla całego repozytorium oraz pętla samodoskonalenia nie występują w żadnym z badanych rozwiązań.

Do oddzielnej luki w literaturze należy problem plateau (zatrzymania postępu) w debacie wielu agentów, opisany na konferencji ICLR 2026 w benchmarku M3MAD-Bench. Większość systemów wielomodelowych traktuje jednomyślną zgodę panelu jako potwierdzenie poprawności. Empirycznie jednomyślność bez głosu odrębnego jest statystycznie rzadka i zwykle wskazuje na kaskadę pewności siebie wewnątrz panelu. Zagadnienie to omawia rozdział 6.4.

## 3. Architektura blast

### 3.1. Układ repozytorium

Framework dystrybuowany jest jako konfiguracja projektu, czyli zestaw plików tekstowych umieszczonych w określonych miejscach drzewa katalogów. Projekt z włączonym blast ma następującą strukturę katalogów najwyższego poziomu: katalog .blast zawiera konstytucję, ustawienia, bazę wiedzy, warstwę kierującą oraz katalog poszczególnych specyfikacji; katalog .claude zawiera komendy, agentów wykonawczych, mechanizmy kontrolne, mosty do modeli zewnętrznych oraz skrypty pomocnicze; pliki najwyższego poziomu (CLAUDE.md, README.md, MANIFEST.md, .env.example) zawierają instrukcje dla modelu, dokumentację oraz szablony konfiguracji.

Podział jest celowy. Katalog .blast pełni rolę pamięci długoterminowej projektu, katalog .claude pełni rolę środowiska wykonawczego agentów. Rozdzielenie pozwala zmienić narzędzie agentowe bez przepisywania specyfikacji i pamięci projektu, oraz pozwala uaktualniać framework dotycząc wyłącznie strony środowiska wykonawczego.



### 3.2. Konstytucja i warstwa kierująca

Plik `.blast/CONSTITUTION.md` jest wiążącym dokumentem governance. Koduje jednaście artykułów porządkujących pracę systemu. Każdy artykuł obejmuje jeden akapit intencji oraz jeden akapit mapowania operacyjnego. Artykuły opisują to, co w systemie pozostaje stałe. Warstwa kierująca (pliki `product.md`, `tech.md`, `structure.md`, `INVENTORY.md`) operacjonalizuje artykuły i może być aktualizowana w trakcie pracy nad poszczególnymi specyfikacjami bez nowelizacji konstytucji. W przypadku konfliktu pomiędzy plikiem warstwy kierującej a artykułem konstytucji, pierwszeństwo otrzymuje artykuł.

Podział funkcjonuje analogicznie do hierarchii prawnej (Konstytucja, ustawy, rozporządzenia) i jest zgodny z koncepcją konstytucyjnej sztucznej inteligencji rozwijaną przez firmę Anthropic. Różnica polega na tym, że blast operacjonalizuje artykuły poprzez konkretne mechanizmy egzekwowania (mechanizmy kontrolne dla artykułu dziesiątego o determinizmie, rejestr komponentów dla artykułu siódmego o niepowtarzaniu rozwiązań, wyzwalacze debat dla artykułu trzeciego o wielomodelowej walidacji), zamiast polegać na tym, że konstytucję każdorazowo przeczyta agent jako wskazówkę.

### 3.3. Maszyna stanów pojedynczej specyfikacji

Każda specyfikacja posiada plik `spec.json` zawierający maszynę stanów. Plik gromadzi nazwę funkcjonalności, fazę bieżącą, status (aktywna, wdrożona, ewoluująca, wycofana), język artefaktów, słownik akceptacji dla każdej fazy produktywnej, sygnały kontekstowe (poziom złożoności, krytyczność bezpieczeństwa, tryb prywatności), listę dostarczonych komponentów oraz listę zależności od innych specyfikacji.

Pole akceptacji dla każdej fazy jest deterministyczną bramką czytaną przez mechanizm kontrolny `blast-approval-gate.py` opisany w rozdziale siódmym. Pola złożoności i krytyczności bezpieczeństwa zasilają heurystyki uruchamiające automatycznie fazy walidacji. Pole trybu prywatności, ustawione na wartość `local-only` (tylko lokalnie), czytane jest przez mechanizm `blast-privacy-gate.py` i blokuje każde wywołanie zewnętrznego modelu językowego dla danej specyfikacji.

### 3.4. Granice determinizmu

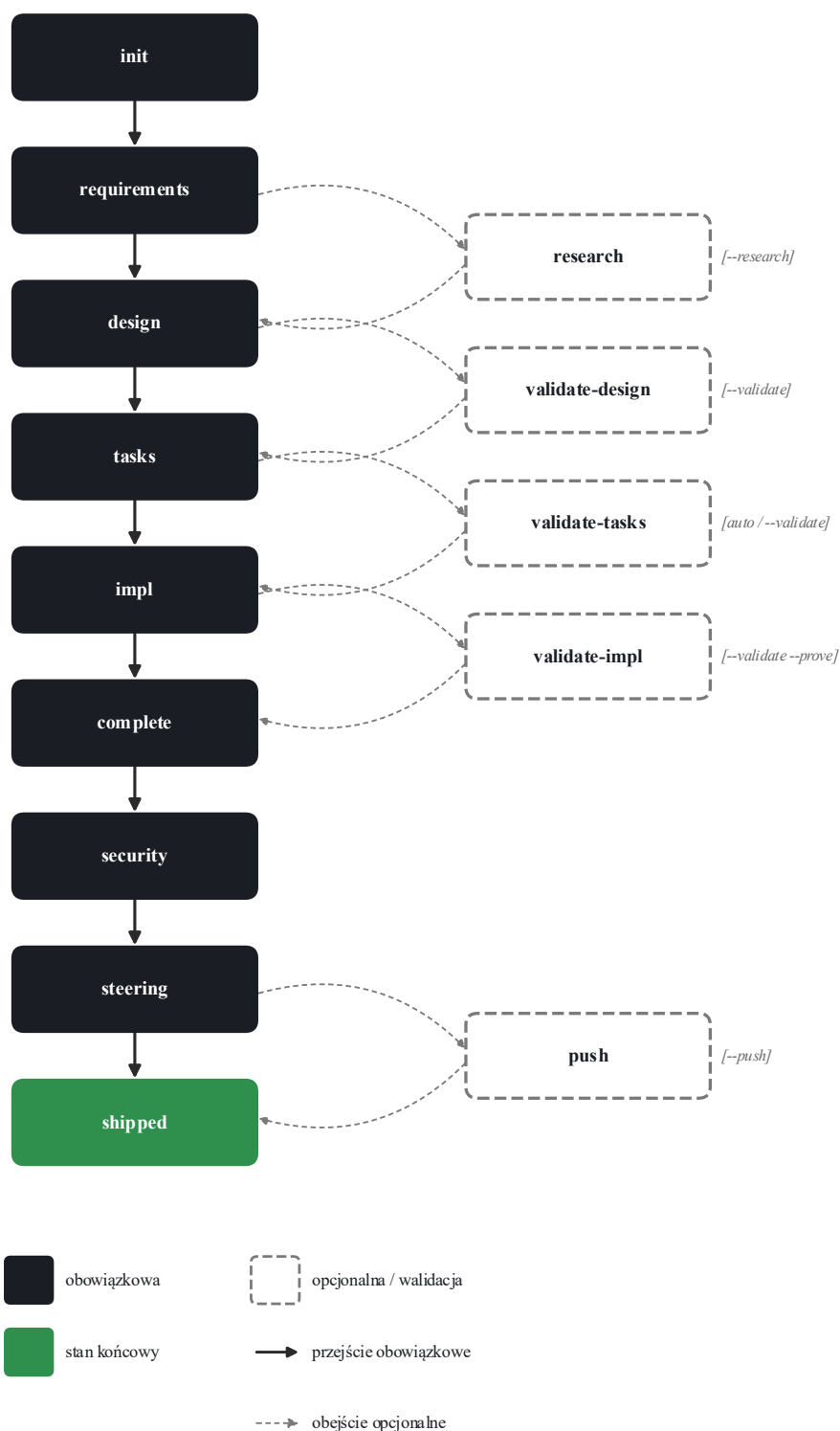
Trzy miejsca w systemie blast pozostają celowo poza domeną decyzji modelu, ponieważ modele językowe okazały się w tych miejscach niewiarygodne. Bramki akceptacji: mechanizm kontrolny czyta plik spec.json i kończy działanie kodem wyjścia 2, jeśli wymagana akceptacja brakuje. Agent w takim przypadku nie zostaje uruchomiony. Bramka prywatności: drugi mechanizm kontrolny czyta tryb prywatności i blokuje wszystkie wywołania zewnętrzne dla specyfikacji oznaczonej jako lokalna. Decyzja kierująca w komendach walidacyjnych: tak zwany wzorzec orchestrator (orkiestratora) umieszcza decyzję o uruchomieniu debaty wewnątrz logiki komendy, gdzie model ma wąski kontekst i jedną decyzję do podjęcia, zamiast wewnątrz agenta walidacyjnego, gdzie konkurowałaby z pracą walidacyjną.

Zasada ogólna: jeżeli debugowanie systemu sprowadza się do pytania, dlaczego agent zdecydował coś pominąć, decyzja powinna zostać przesunięta do deterministycznej bramki. Koszt napisania mechanizmu kontrolnego jest niewielki, koszt niewiarygodności modelu compounduje się wraz z liczbą wywołań.

## 4. Sekwencja faz pracy

### 4.1. Graf przejść

Fazy obowiązkowe układają się w liniowy szkielet o ośmiu przystankach: inicjalizacja (init), specyfikacja wymagań (requirements), projekt techniczny (design), rozpisanie zadań (tasks), implementacja (impl), zamknięcie (complete), audyt bezpieczeństwa (security), synchronizacja pamięci projektu (steering). Pomiedzy poszczególne pary faz wpinają się fazy opcjonalne. Pomiedzy specyfikację wymagań a projekt techniczny: faza badawcza (research). Po projekcie technicznym, po rozpisaniu zadań oraz po implementacji: trzy fazy walidacji (validate-design, validate-tasks, validate-impl). Schemat zależności pomiędzy fazami przedstawia rysunek 1.



Rysunek 1. Sekwencja faz pracy systemu blast wraz ze wskazaniem punktów wstawiania faz opcjonalnych. Fazy obowiązkowe (czarne, wypełnione) tworzą szkielet liniowy biegnący od inicjalizacji do shipowania. Fazy opcjonalne i fazy walidacji (białe, obramowane przerywanym konturem) wstawiają się pomiędzy odpowiednie fazy obowiązkowe pod warunkiem przekazania właściwej flagi (research, validate, push).

Flaga `--auto` komendy `/blast:full` uruchamia każdą fazę bez interakcji z użytkownikiem pomiędzy fazami. Flaga `--validate` aktywuje wszystkie trzy fazy walidacyjne. Flaga `--research` aktywuje fazę badawczą. Flaga `--push` dodaje na końcu sekwencji utworzenie commita oraz wysłanie zmian do zdalnego repozytorium git.

## 4.2. Bramki akceptacji

Pomiędzy fazami produktywnymi (specyfikacja wymagań, projekt techniczny, rozpisanie zadań) działa bramka akceptacji egzekwowana przez mechanizm kontrolny na poziomie środowiska uruchomieniowego. Agent fazy projektu wymaga zaakceptowanej specyfikacji wymagań, agent fazy zadań wymaga zaakceptowanego projektu, agent fazy implementacji wymaga zaakceptowanej listy zadań.

Istnieje pięć ścieżek pominięcia bramki, wszystkie jawne. Pierwsza: typ podagenta nie należy do trzech produktywnych (na przykład walidacja, audyt bezpieczeństwa). Druga: w treści odpowiedzi kierowanej do podagenta znajduje się dokładny zapis `Auto-approve: true` (taki zapis dodawany jest przez komendy uruchomione z flagą `-y`). Trzecia: typ podagenta to `spec-tiny-agent` (przepływ skompresowany dla bardzo małych funkcjonalności). Czwarta: pole `tiny` w specyfikacji ma wartość `true`. Piąta: agent zostaje przepuszczony cicho po spełnieniu jednego z warunków powyżej.

Jeżeli żaden warunek pominięcia nie zachodzi, a poprzednia faza nie jest zaakceptowana, mechanizm kontrolny zwraca kod wyjścia 2 i agent nie zostaje uruchomiony. Komenda wyświetla sugestię następnego kroku, na przykład: `uruchom /blast:approve {nazwa-funkcjonalności} requirements`.

## 4.3. Wykaz komend

System wystawia trzydzieści jeden komend wywoływanych przez użytkownika, pogrupowanych w cztery zbiory. Zbiór pierwszy obejmuje komendy fazy potoku (10 pozycji: `init`, `requirements`, `research`, `design`, `tasks`, `impl`, `complete`, `security`, `steering`, `steering-custom`). Zbiór drugi obejmuje fazy walidacji (4 pozycje: `validate-gap`, `validate-design`, `validate-tasks`, `validate-impl`). Zbiór trzeci obejmuje cykl życia (4 pozycje: `approve`, `evolve`, `deprecate`, `status`). Zbiór czwarty obejmuje kompozycje i komendy meta (13 pozycji: `full`, `quick`, `tiny`, `debate`, `review`, `learn`, `lint`, `graph`, `drift`, `telemetry`, `ping-llm`, `push`, `help`). Pełny wykaz wraz z argumentami i jednoliniowym opisem zachowania znajduje się w dodatku A.

## 4.4. Fazy opcjonalne

Fazy opcjonalne uzasadniają swoje istnienie wyłapywaniem realnych defektów pominiętych przez fazy obowiązkowe. W trakcie testowania systemu blast na funkcjonalności rate-limited-http-client (klient http z ograniczeniem częstotliwości) wykryto sześć defektów wykrytych wyłącznie przez fazy walidacji: niezgodność dekoratora `cached_property` z dekoratorem `dataclass(frozen=True)` prowadząca do błędu wykonania, użycie funkcji `asyncio.get_event_loop().time` jako wartości domyślnej argumentu (prowadzące do problemów w nowszych wersjach języka Python), niedoprecyzowanie kontraktu kolejkowania w klasie `AsyncTokenBucket`, naruszenie wymagania o nazewnictwie metod w wariancie asynchronicznym, brak udokumentowania ryzyka SSRF (Server-Side Request Forgery, czyli sfałszowanego żądania po stronie serwera), wyciek wartości parametrów zapytania do logów (klasyfikacja CWE-200). Każdy z wymienionych defektów został pominięty przez pojedynczego agenta walidacyjnego pracującego w trybie samodzielnym, a wykryty przez panel wielomodelowy. Z tego powodu flaga `--validate` stanowi rekomendowane ustawienie domyślne dla każdej specyfikacji oznaczonej jako wysokiej złożoności lub krytyczna z punktu widzenia bezpieczeństwa.

## 5. System person

### 5.1. Uzasadnienie

Każdy agent fazy posiada nadane imię oraz jednoakapitowy opis roli umieszczony przed jakąkolwiek instrukcją techniczną. Decyzja o nadawaniu agentom imion jest świadomym elementem projektowym, nie ozdobą. Trzy powody tego rozwiązania.

Po pierwsze, separacja ról. Stwierdzenie Atlas projektuje jest krótsze i wyraźniejsze niż stwierdzenie agent obecnie wykonujący fazę projektu. Gdy panel walidacyjny musi odnieść się do treści zaakceptowanej w fazie projektu, czyni to jednoznacznie poprzez imię.

Po drugie, mechanizm wykrywania własnej stronniczości. Każda persona posiada zapisaną słabość, na przykład: Atlas ma tendencję do nadmiernej elegancji architektury kosztem pragmatyzmu. Agenci są instruowani, aby etykietować własną stronniczość w sposób jawny, gdy zauważą u siebie ten dryf. Mechanizm wytwarza ścieżkę audytu o postaci: Atlas-bias:

odrzuca abstrakcję adaptera transportowego jako wytraconą poza zakres potrzeby. Wycofuje sugestię.

Po trzecie, głos charakterystyczny dla każdej fazy. Persona ma udokumentowany styl: Scribe pisze wymagania w notacji EARS dosłownie, Forge pracuje rytmem czerwone, zielone, refaktoryzacja, Pragmatist zadaje pytanie czy to zadanie zarabia na swoje miejsce w planie. Charakterystyczny głos utrzymuje wyniki poszczególnych faz odróżnialne od siebie, co liczy się przy czytaniu specyfikacji po kilku miesiącach.

## 5.2. Obsada

System posiada osiemnaście nazwanych person odpowiadających poszczególnym fazom oraz cztery role pełnione w trakcie debaty wielomodelowej. Tabela 1 podaje pełną obsadę wraz z odpowiadającym podagentem oraz modelem językowym domyślnie używanym.

*Tabela 1. Obsada person systemu blast.*

| Faza                       | Persona  | Podagent                | Model  |
|----------------------------|----------|-------------------------|--------|
| Specyfikacja wymagań       | Scribe   | spec-requirements-agent | Haiku  |
| Faza badawcza              | Oracle   | research-spike-agent    | Sonnet |
| Projekt techniczny         | Atlas    | spec-design-agent       | Opus   |
| Rozpisanie zadań           | Loom     | spec-tasks-agent        | Haiku  |
| Implementacja              | Forge    | spec-tdd-impl-agent     | Sonnet |
| Specyfikacja skompresowana | Sprint   | spec-tiny-agent         | Haiku  |
| Ewolucja funkcjonalności   | Delta    | spec-evolve-agent       | Sonnet |
| Zamknięcie                 | Ledger   | spec-complete-agent     | Haiku  |
| Wycofanie                  | Curator  | spec-deprecate-agent    | Haiku  |
| Analiza luki               | Bridge   | validate-gap-agent      | Sonnet |
| Walidacja projektu         | Crucible | validate-design-agent   | Sonnet |

| Faza                    | Persona      | Podagent                    | Model       |
|-------------------------|--------------|-----------------------------|-------------|
| Walidacja zadań         | Pragmatist   | validate-tasks-agent        | Sonnet      |
| Walidacja implementacji | Auditor      | validate-impl-agent         | Sonnet      |
| Wykrywanie odchyleń     | Tracker      | drift-agent                 | Sonnet      |
| Przegląd kodu           | Compass      | review-agent                | Sonnet      |
| Audyt bezpieczeństwa    | Sentinel     | security-audit-agent        | Opus        |
| Pamięć projektu         | Cartographer | steering-agent              | Sonnet      |
| Pamięć rozszerzona      | Specialist   | steering-custom-agent       | Haiku       |
| Debata: autor           | Author       | debate-author               | Sonnet      |
| Debata: krytyk          | Critic       | debate-critic / critic-opus | Sonnet/Opus |
| Debata: sędzia          | Judge        | debate-judge                | Haiku       |
| Debata: agregator       | Aggregator   | debate-aggregator           | Haiku       |

### 5.3. Mechanizm wykrywania własnej stronniczości w praktyce

Z opisu agenta projektowego (plik `agents/blast/design.md`): masz tendencję do nadmiernej abstrakcji, sięgasz po porty transportowe i klasy bazowe, gdy wystarczyłaby zamknięta struktura danych. Gdy zauważysz, że dodajesz warstwę ponieważ jest ładna, oznacz to wyraźnie zwrotem o postaci: Atlas-bias: dodaję {abstrakcja} z powodu {uzasadnienie}. Wycofuję, ponieważ struktura danych z trzydziestoma liniami powtórzenia jest mniejszą zmianą niż koszt utrzymania abstrakcji.

Wzorzec ujawnił obserwowalne empirycznie zachowanie podczas jednego z testów dymnych. Agent projektu naszkicował adapter blokady asynchronicznej, czyli synchroniczne wejście i wyjście opakowujące blokadę `asyncio.Lock`, w celu współdzielenia obiektu `RequestTracker` pomiędzy ścieżką synchroniczną i asynchroniczną. Panel walidacji projektu wskazał brak technicznej poprawności tego rozwiązania, a w werdykcie odnotowano wprost, że samokontrola stronniczości nie zadziałała wystarczająco mocno. Naprawa polegała na

rozdzieleniu komponentu na dwie osobne klasy z trzydziestoma liniami powtórzonego kodu, a opis słabości osoby został zaktualizowany o ten przykład.

## 5.4. Persona a agent

Persona nie jest tożsama z agentem. Rozróżnienie ma znaczenie. Agent stanowi definicję podagenta w środowisku Claude Code (plik Markdown z metadanymi nagłówka oraz instrukcją). Persona stanowi nazwaną tożsamość, którą może nosić jeden lub więcej agentów. Cztery podagenci debaty (Author, Critic, Judge, Aggregator) działają wewnątrz orkiestracji komendy `/blast:debate` i są uruchamiane przez tę komendę, nie wywoływane bezpośrednio. Nazewnictwo `person` jest przyjazne dla człowieka i pełni funkcję porządkującą dla użytkownika. Nazewnictwo agentów (na przykład `spec-requirements-agent`) jest tym, do czego adresuje wywołania mechanizm `Task` w środowisku Claude Code.

## 6. Walidacja wielomodelowa

### 6.1. Uzasadnienie podejścia

Walidacja prowadzona przez pojedynczy model ma znany tryb porażki: ten sam model wytworzył zarówno artefakt, jak i krytykę. Stronniczości obecne w artefakcie są stronniczościami obecnymi w krytyce. Dwa empiryczne przykłady pochodzące z testów własnych blast.

Pierwszy: naruszenie wymagania o nazewnictwie metod asynchronicznych. Agent implementacyjny wygenerował klasę klienta asynchronicznego, w której metody nosiły nazwy `get`, `post`, `put`, `delete`, `close`. Wymagania wprost przewidywały dla wariantu asynchronicznego nazwy z prefiksem (`aget`, `apost`, `aput`, `adelete`). Pojedynczy agent walidacji w pierwszym przebiegu pominął to naruszenie. Panel HYBRID (autor Sonnet, krytyk Qwen, agregator Haiku) wykrył je, ponieważ krytyk Qwen, wytrenowany na innym korpusie tekstów, porównał kod z wymaganiami i oznaczył naruszenie kontraktu.

Drugi: niezgodność dekoratorów. Agent projektowy zaproponował klasę `Response` z dekoratorami `dataclass(frozen=True)` oraz `cached_property` na metodach `.text` i `.json`. Pierwsze wywołanie metody `.text` próbowałoby zapisać wynik do słownika instancji, co przy zamrożonej strukturze danych powoduje wyjątek `FrozenInstanceError`. Pojedynczy agent walidacji



projektu sygnalizował układ klasy, ale nie zidentyfikował błędu wykonania. Panel JURY\_3\_FLASH3 (Opus, Qwen, Gemini) zgłosił błąd jednomyślnie.

Hipoteza stojąca za wielomodelową walidacją: różnorodność korpusów treningowych wyłapuje różnorodność błędów. Hipoteza znajduje empiryczne wsparcie w przedstawionych obserwacjach oraz w szerszej literaturze poświęconej metodom zespołowym.

## 6.2. Kompozycje paneli

Plik `llm-routing.md` definiuje dwie kompozycje. HYBRID (panel dwuosobowy) używana jest dla walidacji implementacji oraz walidacji zadań. Skład: Sonnet jako autor, Qwen jako krytyk, Haiku jako agregator. JURY\_3\_FLASH3 (panel trzyosobowy z naciskiem na model klasy Flash) używana jest dla walidacji projektu, audytu bezpieczeństwa oraz przeglądu kodu w warunkach wysokiej krytyczności. Skład: Opus jako pierwszy juror, Qwen jako drugi juror, Gemini Flash jako trzeci juror, Haiku jako agregator.

Połączenia są jawne. Każdy juror to albo podagent (uruchamiany przez mechanizm Task, działa we własnym kontekście), albo wywołanie narzędzia zewnętrznego za pośrednictwem mostu MCP (Model Context Protocol, czyli protokół kontekstu modelu). Jeżeli mechanizm wywołania jurora jest niedostępny, na przykład gdy klucz `API GEMINI_API_KEY` nie został ustawiony, juror jest pomijany i ten fakt jest logowany. System nie symuluje brakującego jurora innym agentem występującym w roli zastępczej. Agregator zapisuje degradację w polu `JUROR_DEGRADATIONS` koperty werdyktu.

## 6.3. Mechanika protokołu B

Protokół B to protokół głosowania panelu. Schemat prawidłowej dyspozycji: orkiestrator wystawia wszystkie wywołania jurorów w jednej wiadomości, dzięki czemu środowisko Claude Code rozpisuje je współbieżnie. Każdy juror zapisuje swój blok werdyktu do wspólnego notatnika roboczego umieszczonego w pliku `debates/{topic}.md` w katalogu specyfikacji. Agregator (model Haiku) czyta notatnik, zlicza głosy, zachowuje opinie odrębne dosłownie i emituje końcową kopertę werdyktu.

Przykład końcowej koperty werdyktu z niedawnego audytu bezpieczeństwa zawiera następujące pola: `VERDICT` (werdykt: `WARN`), `BLOCKING` (czy wynik blokuje pipeline: `false`), `FINDINGS` (liczba ustaleń: 3), `DISSENT_COUNT` (liczba opinii odrębnych: 0), `CONSENSUS_REVIEW_RECOMMENDED` (rekomendacja dodatkowej weryfikacji: `true`),

JUROR\_DEGRADATIONS (lista jurorów pominiętych: brak), NEXT\_ACTIONS (lista akcji następnych).

## 6.4. Zabezpieczenie przed kaskadą konsensusu

Benchmark M3MAD-Bench (ICLR 2026) ustalił empirycznie, że debata wieloagentowa może osiągać plateau (zatrzymanie postępu) i dawać się zwieść myłacemu konsensusowi. W warunkach, w których dwóch z trzech jurorów z pewnością przyjmuje błędną odpowiedź, juror trzeci ze statystyczną tendencją ulega presji zamiast zgłaszać opinię odrębną. Konsekwencja: werdykt o pełnej zgodzie bez głosu odrębnego, wydany na realnym artefakcie, jest z punktu widzenia wnioskowania o jakości tego artefaktu podejrzany, a nie potwierdzający. Realne projekty prawie zawsze posiadają choć jedną możliwość poprawy. Pełna zgoda zwykle wskazuje na kaskadę pewności siebie wewnątrz panelu.

Agregator systemu blast realizuje jawne zabezpieczenie w postaci następującej reguły: jeżeli liczba opinii odrębnych wynosi zero, werdykt mieści się w kategoriach PASS lub WARN, oraz liczba ustaleń krytycznych wynosi zero, agregator ustawia w kopercie werdyktu pole `CONSENSUS_REVIEW_RECOMMENDED` na wartość `true` i rekomenduje rundę adwokata diabła (Protocol D) lub przegląd przez człowieka.

Zabezpieczenie nie jest automatycznym powtórzeniem debaty. Powtórzenie byłoby kosztowne i niekoniecznie przełamałoby kaskadę. Zabezpieczenie ma postać flagi przekazywanej człowiekowi sprawdzającemu, który może uruchomić Protocol D (jurorzy zobowiązani do znalezienia co najmniej trzech słabości) lub zaakceptować werdykt świadom, że nie został on skalibrowany. Według wiedzy autora system blast jest jedynym publicznym rozwiązaniem klasy SDD operacjonalizującym ustalenie M3MAD-Bench.

## 6.5. Kierowanie zapytań do tańszych modeli

Powyższe kompozycje celują w różne poziomy kosztowe. Tabela 2 podaje przybliżone koszty per faza.

*Tabela 2. Koszt jednostkowy per faza (orientacyjny, dolar amerykański).*

| Faza (persona)                | Kompozycja        | Koszt       |
|-------------------------------|-------------------|-------------|
| specyfikacja wymagań (Scribe) | samodzielny Haiku | około 0,001 |

| Faza (persona)                  | Kompozycja                       | Koszt           |
|---------------------------------|----------------------------------|-----------------|
| rozpisanie zadań (Loom)         | samodzielny Haiku                | około 0,002     |
| projekt techniczny (Atlas)      | samodzielny Opus                 | około 0,05      |
| implementacja: zadanie proste   | Qwen przez most MCP              | 0,00 (lokalnie) |
| implementacja: zadanie złożone  | samodzielny Sonnet               | 0,10 do 0,30    |
| walidacja zadań (Pragmatist)    | HYBRID                           | około 0,12      |
| walidacja projektu (Crucible)   | JURY_3_FLASH3                    | około 0,50      |
| audyt bezpieczeństwa (Sentinel) | JURY_3_FLASH3 (zawsze)           | około 1,00      |
| przegląd kodu (Compass)         | JURY_3_FLASH3 dla auth/płatności | około 1,00      |

Strategia lokalności w fazie implementacji ma istotne konsekwencje finansowe. Dla funkcji czystych, struktur danych oraz pomocników bezstanowych, model Qwen3-coder pracujący na lokalnej karcie graficznej RTX 5090 produkuje działający kod w czasie rzędu trzydziestu sekund przy zerowym koszcie krańcowym. Agent fazy implementacji klasyfikuje zadania na początku fazy i kieruje proste zadania do modelu lokalnego, złożone (asynchroniczność, maszyny stanów, zagadnienia przekrojowe) eskalują do modelu Sonnet.

Limity kosztu na fazę egzekwowane są w trybie miękkim. Gdy faza przekracza limit kosztu, werdykt zawiera w polu NEXT\_ACTIONS notatkę debate truncated (debata przycięta), zamiast wymuszać zatrzymanie potoku. Unika to najgorszego trybu porażki (cichy narost kosztów) bez popadania w drugi najgorszy (twarde zatrzymanie blisko ukończonej debaty).

## 6.6. Tryb prywatności

Ustawienie pola privacy w pliku spec.json na wartość local-only blokuje każde wywołanie zewnętrznego modelu poprzez mechanizm kontrolny blast-privacy-gate.py. Jest to mechanizm uruchamiany przed każdym wywołaniem narzędzia (PreToolUse), kończący działanie kodem wyjścia 2 dla każdego pasującego wzorca nazwy narzędzia (między innymi: WebSearch, WebFetch, mcp\_plugin\_\*, mcp\_blast-llm-bridge\_ask\_gemini\_\*). Wywołania lokalne pozostają dozwolone.

Gdy tryb prywatności jest aktywny, kompozycje paneli degradują do wariantu wyłącznie lokalnego. Panel HYBRID otrzymuje skład jurorów Qwen3.6 oraz Qwen3-coder, agregatorem zostaje Qwen3.6. Panel JURY\_3\_FLASH3 pracuje analogicznie, opcjonalnie z modelem gpt-oss jako trzecim jurorem, jeżeli ten został zainstalowany lokalnie. Limit kosztu spada do zera. Mechanizm działa na poziomie środowiska uruchomieniowego, nie poprzez instrukcję kontekstową. Specyfikacja w trybie prywatności nie może przypadkowo wyciec do modelu w chmurze przez błąd interpretacji ze strony agenta.

## **7. Mechanizmy egzekwowania determinizmu**

### **7.1. Trzy mechanizmy kontrolne**

System rejestruje trzy mechanizmy kontrolne w pliku `.claude/settings.json`. Pierwszy: `blast-approval-gate.py` (PreToolUse na wywołaniach Agent oraz Task) sprawdza, czy poprzednia faza została zaakceptowana, i blokuje uruchomienie agenta w przeciwnym razie. Drugi: `blast-privacy-gate.py` (PreToolUse dla narzędzi pasujących do wzorca zewnętrznego) blokuje wszelką komunikację zewnętrzną dla specyfikacji oznaczonej jako lokalna. Trzeci: `blast-telemetry.py` (PostToolUse dla każdego wywołania narzędzia) zapisuje telemetrię (czas, koszt, status) do strumienia obserwowalności.

Wydajność każdego mechanizmu mieści się w okolicy piętnastu milisekund (uruchomienie interpretera Pythona oraz operacje wejścia i wyjścia). Wartość ta jest niezauważalna w tempie pracy użytkownika i staje się istotna jedynie przy skali, gdy agenci uruchamiają dziesiątki podagentów w trakcie jednej sesji.

Tryb porażki każdego mechanizmu obsługowany jest defensywnie. Jeżeli pliku `spec.json` nie da się sparsować, mechanizm rejestruje ostrzeżenie i kończy działanie kodem wyjścia 0 (przepuszcza). Decyzja o stroniczości na rzecz nie blokowania pracy oparta jest o założenie, że błędy mechanizmów to wady frameworku, nie kary dla użytkownika.

### **7.2. Uzasadnienie ogólne**

Argument ogólnego przeznaczenia: gdy debugowanie sprowadza się do pytania, dlaczego agent zdecydował coś pominąć, decyzja powinna zostać przesunięta do deterministycznej bramki. Koszt napisania mechanizmu kontrolnego jest niewielki. Koszt niewiarygodności modelu komponentuje się wraz z liczbą wywołań.

Argument szczegółowy: w trakcie rozwoju systemu blast faza walidacji zadań została znaleziona jako cicho pomijana w przebiegu `/blast:full --auto --validate`, mimo że specyfikacja nosiła oznaczenia wysokiej złożoności i krytyczności bezpieczeństwa. Przyczyna pierwotna: orkiestrator interpretował fazę jako opcjonalną, ponieważ jej nagłówek zawierał słowo `conditional` (warunkowa). Naprawa polegała na wstrzyknięciu zadania walidacji do listy `todo` oraz przeformułowaniu nagłówka. Naprawa miała charakter deterministyczny, nie polegała na prośbie kierowanej do modelu o uważność.

### 7.3. Wzorzec orkiestratora

Pokrewnym ruchem w stronę determinizmu jest tak zwany wzorzec orkiestratora dla komend walidacji. Zamiast pozwolić agentowi walidacji zdecydować, czy uruchomić debatę (wcześniejsze wersje systemu blast tak działały, agent ignorował własną logikę kierującą), komenda walidacji deterministycznie wykonuje pięć kroków. Krok pierwszy: parsuje argumenty wywołania, wyszukuje flagę `--no-debate`. Krok drugi: czyta plik `llm-routing.md` i lokalizuje konfigurację `debate_config` dla danej fazy. Krok trzeci: oblicza decyzję FIRE (uruchom debatę) lub SKIP (pomiń debatę) zgodnie z udokumentowanym algorytmem. Krok czwarty: emituje na wyjściu dosłowną linię o postaci `Routing: <FIRE/SKIP> — <uzasadnienie>`. Krok piąty: rozgałęzia się do komendy `/blast:debate` albo do agenta walidacji w trybie samodzielnym.

Agent otrzymuje węższy zakres odpowiedzialności (waliduj, nie decyduj o trasie). Linia kierowania pozostaje audytowalna w zapisie sesji. Wzorzec udokumentowany jest w punkcie 3.4 pliku `CLAUDE.md` i był istotnym refaktorem w połowie 2026 roku.

## 8. Pamięć projektu i samodoskonalenie

### 8.1. Warstwa kierująca jako pamięć

Katalog `.blast/steering` pełni rolę pamięci długoterminowej projektu, ładowanej automatycznie przy każdym wywołaniu agenta. Cztery pliki podstawowe (`product.md`, `tech.md`, `structure.md`, `INVENTORY.md`) wraz z opcjonalnymi suplementami (`llm-routing.md`, `cost-policy.md`, `RESEARCH.md`) tworzą kontrakt pomiędzy człowiekiem a agentem: zbiór twierdzeń o projekcie, które agent powinien znać bez bycia każdorazowo o nich informowanym.

Warstwa kierująca aktualizowana jest trzema mechanizmami. Pierwszy: rozruch w nowym projekcie (`bootstrap`). Komenda `/blast:steering` wykrywa świeżą inicjalizację za pomocą

znacznika `BLAST_STUB` i uruchamia tryb pytań do użytkownika. Pięć do siedmiu pytań pozwala wypełnić warstwę kierującą odpowiedziami użytkownika, zamiast wnioskować je z plików frameworku. Drugi: synchronizacja po wdrożonej funkcjonalności. Komenda `/blast:complete` uruchamia retrospekcję, która proponuje wnioski (problemy, wymagania niezmiennikowe, antywzorce) do włączenia do plików `tech.md` lub `product.md`. Agent stosuje sprawdzenie najbliższego sąsiada: jeżeli proponowany wniosek jest bliski istniejącej regule, doprecyzowuje istniejącą zamiast dodawać nowe ujęcie. Trzeci: wykrywanie odchyleń. Komenda `/blast:drift` oraz tryb synchronizacji agenta warstwy kierującej oznaczają sekcje warstwy kierującej, które przestały odpowiadać kodowi (na przykład lista dozwolonych zależności, w której brakuje biblioteki obecnie importowanej w trzech modułach).

## 8.2. Baza wiedzy i kuratorska SOTA

Katalog `.blast/knowledge` stanowi długoterminową wiedzę zewnętrzną projektu. Zawiera cztery podkatalogi. Decisions: rejestr decyzji architektonicznych (Architectural Decision Records, ADR). Research: wyniki fazy badawczej dla poszczególnych funkcjonalności w postaci destylatu. References: zachowane specyfikacje API, notatki o bibliotekach, znalezione pułapki implementacyjne. Sota: kuratorskie rekomendacje stanu wiedzy dla poszczególnych domen.

Podkatalog sota stanowi novum względem badanych frameworków. Każdy plik (na przykład `http-clients.md`, `async-patterns.md`, `database-orm.md`) zawiera datę ostatniego odświeżenia, rekomendowane wybory wraz z uzasadnieniem, antywzorce oraz odrzucone alternatywy, próg wymagający ponownego badania. Persona Pragmatist (faza walidacji zadań) konsultuje pliki sota przed sugerowaniem alternatyw bibliotecznych, co eliminuje tryb porażki, w którym model wytrenowany na danych z 2023 roku rekomenduje deprecjonowaną bibliotekę. Pliki starsze niż sześć miesięcy oznaczane są przez komendę `/blast:learn --refresh-sota` do ponownego zbadania.

## 8.3. Pętla samodoskonalenia

Komenda `/blast:learn` jest agregatorem czterotrybowym. Tryb `--lessons` wyciąga powtarzające się wzorce z zakończonych specyfikacji i włącza je do warstwy kierującej. Tryb `--calibrate` porównuje koszt szacowany z faktycznym dla każdej fazy i aktualizuje wpisy w pliku `llm-routing.md`. Tryb `--routing` rejestruje częstotliwość uruchamiania kompozycji oraz zdarzenia

degradacji. Tryb `--refresh-sota` audytuje pliki sota pod kątem wieku. Tryb `--all` uruchamia wszystkie cztery.

Pętla uruchamia się automatycznie co pięć wdrożonych specyfikacji (licznik w pliku `.blast/shipped-counter`), co czyni utrzymywanie warstwy kierującej oraz konfiguracji kierowania zapytań skalibrowanymi w miarę starzenia projektu zadaniem o niskim koszcie. Jest to jedyna pętla samodoskonalenia w badanych frameworkach SDD. Pozostałe rozwiązania polegają na ręcznym zarządzaniu odchyleniami konfiguracji.

## 8.4. Rejestr komponentów wspólny dla całego repozytorium

Plik `.blast/steering/INVENTORY.md` zawiera tabelę komponentów wypełnianą przez komendę `/blast:complete`. Schemat tabeli: komponent, typ, funkcjonalność źródłowa, opis. Przed zaprojektowaniem nowego komponentu, agent fazy projektu (Atlas) sprawdza w INVENTORY istnienie odpowiednika. Dopasowanie ma charakter heurystyczny (agent stosuje semantyczne podobieństwo do opisu), agent jest jednak zobowiązany do uzasadnienia w pliku `design.md`, dlaczego tworzony jest nowy komponent, jeżeli INVENTORY zawiera już zbliżony. Reinwencja wdrożonego komponentu bez uzasadnienia jest sygnalizowana podczas retrospekcji w fazie zamknięcia.

## 9. Cykl życia poza wdrożeniem

Funkcjonalność nie kończy swojego życia w momencie wdrożenia. Cykl trwa dalej.

Komenda `/blast:evolve {nazwa} "<zmiana>"` generuje specyfikację różnicową w katalogu `.blast/specs/{nazwa}/evolutions/{N}-{slug}/`. Specyfikacja różnicowa posiada własną pojedynczą bramkę akceptacji (osobne pliki `evolution.md` oraz `tasks.md`), jest implementowana komendą `/blast:impl` i scalana z powrotem do specyfikacji nadrzędnej przez komendę `/blast:complete` (która kieruje pracę przez krok zerowy w agencie zamknięcia).

Komenda `/blast:deprecate {nazwa} --reason "..."` oznacza wdrożoną funkcjonalność jako wycofaną, generuje przewodnik migracji jeżeli istnieje zamiennik, oraz oznacza specyfikacje zależne w rejestrze komponentów. Status `deprecated` jest honorowany przez pozostałych agentów: nie będą cicho zależeć od wycofanego komponentu.

Komenda `/blast:security {nazwa}` może zostać uruchomiona ponownie po wdrożeniu. Wyzwalacz `always` (zawsze) gwarantuje, że werdykt audytu bezpieczeństwa nigdy nie pochodzi od pojedynczego modelu, nawet poza przepływem `--auto`.

Komenda `/blast:drift {nazwa}` wykrywa odchylenia pomiędzy specyfikacją a kodem, sugerując albo ponowny projekt (specyfikacja błędna), albo refaktoryzację kodu (kod odszedł od specyfikacji).

Jawnie zdefiniowany cykl życia odróżnia blast od większości badanych frameworków, które traktują wdrożenie jako stan terminalny i polegają na pamięci użytkownika dotyczącej tego, co zostało wdrożone.

## 10. Ocena rozwiązania

### 10.1. Macierz porównawcza

Tabela 3 zawiera porównanie systemu blast z trzema rozwiązaniami referencyjnymi (Kiro, GitHub Spec Kit, BMAD-METHOD) w dwudziestu jeden wymiarach technicznych, według stanu na maj 2026 roku. Oznaczenia: TAK (wsparcie pierwszej kategorii), CZĘŚĆ (obecne, ale nie centralne), NIE (nieobecne), implicit (obecne pośrednio).

*Tabela 3. Macierz porównawcza systemów SDD na maj 2026.*

| Wymiar                              | Spec Kit | BMAD     | Kiro     | blast |
|-------------------------------------|----------|----------|----------|-------|
| Trzyfazowy potok artefaktów         | TAK      | TAK      | TAK      | TAK   |
| Faza badawcza                       | NIE      | CZĘŚĆ    | NIE      | TAK   |
| System person                       | implicit | TAK      | NIE      | TAK   |
| Konstytucja jako odrębny dokument   | TAK      | implicit | implicit | TAK   |
| Pamięć projektu (warstwa kierująca) | CZĘŚĆ    | CZĘŚĆ    | CZĘŚĆ    | TAK   |
| Rejestr komponentów (cross-spec)    | NIE      | CZĘŚĆ    | NIE      | TAK   |
| Fazy walidacji                      | NIE      | CZĘŚĆ    | CZĘŚĆ    | TAK   |



| Wymiar                                             | Spec Kit | BMAD  | Kiro     | blast    |
|----------------------------------------------------|----------|-------|----------|----------|
| Walidacja wielomodelowa                            | NIE      | NIE   | NIE      | TAK      |
| Wielodostawcze panele (Anthropic, Google, lokalne) | NIE      | CZĘŚĆ | NIE      | TAK      |
| Mechanizmy kontrolne na poziomie SDK               | NIE      | NIE   | TAK      | TAK      |
| Kierowanie zapytań do tańszego modelu              | NIE      | NIE   | NIE      | TAK      |
| Tryb prywatności (lokalny zapas)                   | NIE      | NIE   | NIE      | TAK      |
| Egzekwowanie TDD                                   | implicit | CZĘŚĆ | CZĘŚĆ    | TAK      |
| Cykl życia: ewolucja, wycofanie                    | CZĘŚĆ    | CZĘŚĆ | CZĘŚĆ    | TAK      |
| Pętla samodoskonalenia                             | NIE      | CZĘŚĆ | NIE      | TAK      |
| Kuratorska wiedza SOTA                             | NIE      | CZĘŚĆ | NIE      | TAK      |
| Zabezpieczenie przed kaskadą konsensusu            | NIE      | NIE   | NIE      | TAK      |
| Limity kosztu na fazę                              | NIE      | NIE   | NIE      | TAK      |
| Narzędzie do tworzenia nowych projektów            | TAK      | TAK   | CZĘŚĆ    | TAK      |
| Wsparcie wielu IDE                                 | TAK      | CZĘŚĆ | NIE      | NIE      |
| Widoczność publiczna                               | średnia  | 46k★  | komercja | osobiste |

Z dwudziestu jeden ocenianych wymiarów blast wyprzedza alternatywy w dziewięciu (faza badawcza, walidacja wielomodelowa, panele wielodostawcze, kierowanie zapytań do tańszego modelu, tryb prywatności, głębina faz walidacji, zabezpieczenie przed kaskadą konsensusu, pętla samodoskonalenia, cykl życia). Plasuje się na poziomie porównywalnym z co najmniej jedną alternatywą w ośmiu (potok faz, system person, konstytucja, pamięć projektu, mechanizmy kontrolne, TDD, narzędzie tworzenia projektów, rejestr komponentów). Pozostaje w tyle w dwóch (wsparcie wielu IDE, dystrybucja i społeczność). Wymiary unikalne dla blast (kuratorska SOTA per domena, limity kosztu na fazę) są trudne do bezpośredniego porównania.

## 10.2. Dowody empiryczne

Powyższe przewagi techniczne nie mają charakteru wyłącznie teoretycznego. W trakcie rozwoju systemu blast walidacja wielomodelowa wykryła kilka rzeczywistych defektów, które pojedyncze modele pominęły. Tabela 4 zawiera ich zestawienie.

*Tabela 4. Defekty wykryte przez walidację wielomodelową, pominięte przez pojedynczy model.*

| Defekt                                                                                          | Wykryty przez            | Pominięty przez |
|-------------------------------------------------------------------------------------------------|--------------------------|-----------------|
| Niezgodność <code>cached_property</code> z <code>dataclass(frozen=True)</code> : błąd wykonania | validate-design (JURY_3) | Crucible solo   |
| <code>asyncio.get_event_loop()</code> jako wartość domyślna argumentu                           | validate-design (JURY_3) | Crucible solo   |
| Niedoprecyzowanie kontraktu kolejki w klasie <code>AsyncTokenBucket</code>                      | validate-design (JURY_3) | Crucible solo   |
| Naruszenie REQ 25.2: nazewnictwo metod asynchronicznych                                         | validate-impl (HYBRID)   | Auditor solo    |
| Brak udokumentowania ryzyka SSRF (caller responsibility)                                        | security (JURY_3)        | Sentinel solo   |
| Wyciek wartości parametrów zapytania do logów (CWE-200)                                         | security (JURY_3)        | Sentinel solo   |

Sześć defektów na jednej funkcjonalności, z różnym profilem ryzyka: od zaskoczenia użytkownika błędem wykonania po wyciek danych poufnych do logów. System paneli wielomodelowych zwraca koszty swojego uruchomienia.

Profil wydajnościowy. Pełny przebieg potoku z flagami `--auto --research --validate` dla funkcjonalności o trzydziestu dwóch wymaganiach EARS, jedenastu modułach i dwustu dziewięćdziesięciu pięciu testach trwał około osiemdziesiąt minut. Rozkład czasowy: faza badawcza pięć minut (po naprawie równoległego wyszukiwania w sieci oraz pamięci podręcznej bazy wiedzy, czas spadł z czterdziestu ośmiu minut), faza projektu osiem minut, fazy walidacji łącznie szesnaście minut (sześć, cztery, sześć), faza implementacji trzydzieści osiem minut, pozostałe fazy łącznie dwanaście minut. Dominującą kosztowo fazą jest implementacja. Walidacja oraz badania mieszczą się w budżecie z zapasem.

### 10.3. Pozycja blast w stosunku do stanu wiedzy

Wyprzedza: panele wielodostawcze działające równolegle, zabezpieczenie przed kaskadą konsensusu, tryb prywatności, kierowanie zapytań do tańszego modelu, rejestr komponentów wspólny dla repozytorium, pętla samodoskonalenia, kuratorska wiedza SOTA, cykl życia poza wdrożeniem, limity kosztu na fazę, cztery wyodrębnione fazy walidacji wraz z weryfikacją behawioralną.

Pozostaje na poziomie porównywalnym: kształt potoku faz, system person (BMAD oferuje porównywalną głębię), konstytucja (Spec Kit ma dziewięć artykułów, blast jedenaście), pliki warstwy kierującej, narzędzie tworzenia projektów.

Pozostaje w tyle: wsparcie dla wielu środowisk programistycznych (Spec Kit pracuje z trzema klientami: GitHub Copilot, Claude Code, Gemini CLI; blast wymaga wyłącznie środowiska Claude Code). Dystrybucja i społeczność (BMAD ma 46,2 tysiąca gwiazdek na GitHubie; blast działa w obrębie jednego repozytorium autorskiego). Polerowanie dokumentacji (Spec Kit ma osobną stronę projektu; blast ma plik README.md oraz niniejsze opracowanie).

Asymetria pozostaje spójna: blast inwestuje w głębię techniczną, badane alternatywy inwestują w dystrybucję. Dla indywidualnego praktyka oraz małego zespołu blast jest konkurencyjny w czołówce. Dla dużej organizacji wymagającej wsparcia dostawcy oraz bogatego ekosystemu, alternatywy oferują niższy próg wejścia.

## 11. Ograniczenia i kierunki rozwoju

### 11.1. Sprzężenie z jednym środowiskiem programistycznym

Agenci systemu blast są zapisani w formacie podagentów środowiska Claude Code (pliki Markdown z metadanymi nagłówka deklarującymi nazwę, opis, dostępne narzędzia, model). Adaptacja do innych narzędzi (Cursor, Windsurf, Zed, Aider) wymagałaby przetłumaczenia formatu oraz dostosowania mechanizmów kontrolnych na poziomie SDK (które obecnie polegają na zdarzeniu PreToolUse charakterystycznym dla Claude Code). Zadanie nietrywialne, ale wykonalne. Definicje podagentów są w istocie szablonami odpowiedzi z listą dozwolonych narzędzi, a większość współczesnych środowisk wspierających modele językowe posiada analogiczne prymitywy.

## 11.2. Dystrybucja i pakowanie

Framework rozprawiany jest jako repozytorium szablonowe git. Skrypt `blast init` (napisany wyłącznie w bibliotece standardowej języka Python) klonuje szablon i resetuje stan specyficzny dla projektu autora. Brak instalacji przez menedżery pakietów typu `pip` czy `npm`. Użytkownik musi posłużyć się jednym z dwóch sposobów: wywołanie skryptu przez polecenie `curl` pobierające go z repozytorium, albo lokalne klonowanie repozytorium. Pakowanie dystrybucyjne obniżyłoby próg wejścia.

## 11.3. Plateau debaty

Ustalenie z benchmarku M3MAD-Bench (omówione w punkcie 6.4) jest mitygowane flagą `consensus_review_recommended` w kopercie werdyktu agregatora. Mitygacja wystawia flagę zamiast automatycznego przekierowania w tryb adwokata diabła. W praktyce użytkownik musi przeczytać kopertę werdyktu i podjąć decyzję. Solidniejsze rozwiązanie wymuszałoby automatyczne uruchomienie Protocol D, gdy `CONSENSUS_REVIEW_RECOMMENDED` przyjmuje wartość `true`, akceptując dodatkowy koszt.

## 11.4. Współbieżność w fazie implementacji

Faza implementacji posiada wykonanie współbieżne oparte na falach, oznaczone w pliku `tasks.md` znacznikiem (P) oraz uruchamiane jako podagenty z izolacją `worktree` (osobny katalog roboczy git dla każdego zadania). Mechanizm jest ciężki: każde zadanie z oznaczeniem (P) uruchamia pełnego podagenta w izolowanym katalogu git. Dla zadań prostych (funkcje czyste, struktury danych) jest to nadmiarowe. Pakowanie wielu wywołań Qwen poprzez most MCP w jednej wiadomości byłoby od czterech do ośmiu razy szybsze. Agent fazy implementacji obecnie nie rozróżnia zadań prostych od złożonych przy planowaniu pracy współbieżnej. Pozostaje to na liście prac przyszłych.

## 11.5. Ryzyko ról zastępczych

W wersjach poprzedzających tryb debaty system po cichu odgrywał role nieobecnych jurorów (model Sonnet udający Opus, gdy nie był wpięty żaden podagent oparty na Opus). Zachowanie zostało wykryte i naprawione w trakcie rozwoju. Agregator obecnie wprost zakazuje takich zastępstw i zapisuje je w polu `JUROR_DEGRADATIONS`. Jeżeli jednak przyszła wersja środowiska Claude Code zmieni mechanizm wywoływania podagentów, pokusa odgrywania

zastępstw może powrócić inną drogą. Test regresyjny (debata z jurorom celowo niedostępnym) wzmocniłby zabezpieczenie.

## 11.6. Koszt walidacji dla małych funkcjonalności

Dla funkcjonalności trywialnych (jedna funkcja narzędziowa, schemat konfiguracji) czterofazowy potok walidacji jest nadmiarowy. Komenda `/blast:tiny` istnieje dla takich przypadków (skompresowana specyfikacja, jednoprzebiegowa implementacja, brak debaty). Próg klasyfikujący funkcjonalność jako trywialną jest obecnie subiektywny. Heurystyka oparta o wartości pól `complexity_hint` oraz licznik zadań mogłaby automatycznie kierować małe specyfikacje do trybu skompresowanego.

## 12. Podsumowanie

blast jest frameworkiem rozwoju oprogramowania kierowanego specyfikacją, który operacjonalizuje trzy zasady niespotykane łącznie we współczesnych rozwiązaniach klasy SOTA. Bramki deterministyczne na poziomie środowiska uruchomieniowego (mechanizmy kontrolne dla akceptacji, prywatności, kierowania zapytań), zamiast próśb na poziomie odpowiedzi. Różnorodność wielodostawcza (równoległe panele oceniające z modelami Anthropic, Google oraz lokalnym modelem Qwen, wraz z jawną mitygacją kaskady konsensusu). Świadomość cyklu życia (ewolucja, wycofanie, wykrywanie odchyłeń, oraz pętla samodoskonalenia uruchamiana co pięć wdrożonych specyfikacji).

Cechą wyróżniającą framework jest kombinacja tych decyzji, nie żadna z nich z osobna. Każda pojedyncza decyzja techniczna (nazwane persony, wymagania w notacji EARS, konstytucja jako governance, rejestr komponentów wspólny dla repozytorium, kierowanie zapytań do tańszego modelu) ma analogie w badanych alternatywach (Kiro, GitHub Spec Kit, BMAD-METHOD). Niespotykane jest systematyczne zastosowanie wszystkich z postawą wymuszania determinizmu, która wypycha decyzje poza kontekst modelu, gdy jego zmienność stanowi wąskie gardło.

Według macierzy porównawczej z punktu 10.1, blast plasuje się w czołówce współczesnych frameworków SDD pod względem głębi technicznej. Wyprzedza w obszarach paneli wielodostawczych, trybu prywatności, kierowania zapytań do tańszego modelu, rejestru komponentów, samodoskonalenia oraz cyklu życia; pozostaje na poziomie porównywalnym w

większości pozostałych; pozostaje w tyle pod kątem dystrybucji i pokrycia różnych środowisk programistycznych. Dla indywidualnego praktyka albo małego zespołu pracującego w środowisku Claude Code, blast jest najsilniejszym dostępnym wyborem. Dla większych organizacji albo procesów spoza ekosystemu Claude Code, alternatywy oferują niższy próg wejścia.

## Literatura

Lista pozycji literaturowych odzwierciedla materiały przeszukane w trakcie pracy nad niniejszym opracowaniem. Adresy internetowe pozostawały aktualne na dzień 7 maja 2026.

- [1] GitHub. Spec Kit. Repozytorium projektu. Dostęp: <https://github.com/github/spec-kit>
- [2] Microsoft for Developers. Diving Into Spec-Driven Development With GitHub Spec Kit. Grudzień 2025. Dostęp: <https://developer.microsoft.com/blog/spec-driven-development-spec-kit>
- [3] Amazon Web Services. Kiro: Integrated Development Environment for Spec-Driven Development. Notatki wydania komercyjnego, lipiec 2025. Dostęp: <https://kiro.dev/>
- [4] BMAD-METHOD. Breakthrough Method for Agile AI-Driven Development, wersja 6.6.0. Kwiecień 2026. Dostęp: <https://github.com/bmad-code-org/BMAD-METHOD>
- [5] Fowler, M. Spec-Driven Development is Eating Software Engineering. Marzec 2026. Dostęp: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- [6] Pillitteri, P. Goodbye Vibe Coding: SDD Frameworks. 2026. Dostęp: <https://pasqualepillitteri.it/en/news/158/framework-ai-spec-driven-development-guide-bmad-gsd-ralph-loop>
- [7] Mysore, V. Spec-Driven Development Is Eating Software Engineering: Mapa trzydziestu agentowych frameworków programowania. Medium, marzec 2026.
- [8] Komitet ICLR. M3MAD-Bench: Multi-Model Multi-Agent Debate Benchmark. Materiały z Międzynarodowej Konferencji nad Reprezentacjami Uczenia (ICLR), 2026.
- [9] Anthropic. Building Multi-Agent Systems: When and How to Use Them. 2026. Dostęp: <https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them>

- [10] Zhang, Y. i inni. Agent4Debate: A Multi-Agent Debate Framework. Materiały z Międzynarodowej Konferencji nad Akustyką, Mową i Przetwarzaniem Sygnałów (ICASSP), 2026. Dostęp: <https://github.com/zhangyiqun018/agent-for-debate>
- [11] Anthropic. Claude Code Documentation. Dostęp: <https://docs.claude.com> (sekcje: subagents, hooks, Skill tool, Model Context Protocol).
- [12] Anthropic. Constitutional AI: Harmlessness from AI Feedback. Dostęp: <https://www.anthropic.com/research/constitutional-ai>
- [13] OWASP Foundation. SSRF Prevention Cheat Sheet. Dostęp: [https://cheatsheetseries.owasp.org/cheatsheets/Server\\_Side\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html)
- [14] MITRE Corporation. Common Weakness Enumeration: CWE-117 Improper Output Neutralization for Logs; CWE-200 Information Exposure; CWE-918 Server-Side Request Forgery. Dostęp: <https://cwe.mitre.org/>
- [15] Mavin, A.; Wilkinson, P.; Harwood, A.; Novak, M. Easy Approach to Requirements Syntax (EARS). Materiały z Siedemnastej Międzynarodowej Konferencji IEEE nad Inżynierią Wymagań, 2009.

## Dodatek A. Wykaz komend systemu

Poniżej zestawienie wszystkich trzydziestu jeden komend wywoływanych przez użytkownika, z argumentami oraz jednoliniowym opisem. Pełny opis zachowania każdej komendy dostępny jest pod `/blast:help <nazwa>`.

### A.1. Fazy potoku

| Komenda                             | Argumenty                                                         | Opis                                                                              |
|-------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>/blast:init</code>            | <code>&lt;nazwa&gt;</code> lub <code>--source &lt;plik&gt;</code> | Inicjalizacja nowej specyfikacji z opisu lub dokumentu źródłowego                 |
| <code>/blast:requirements</code>    | <code>&lt;nazwa&gt;</code>                                        | Generowanie wymagań w notacji EARS (persona Scribe)                               |
| <code>/blast:research</code>        | <code>&lt;nazwa&gt; [--deep]</code>                               | Faza badawcza: badanie opcji, porównanie podejść (persona Oracle)                 |
| <code>/blast:design</code>          | <code>&lt;nazwa&gt; [-y]</code>                                   | Generowanie projektu technicznego (persona Atlas)                                 |
| <code>/blast:tasks</code>           | <code>&lt;nazwa&gt; [-y]</code>                                   | Rozpisanie zadań implementacyjnych (persona Loom)                                 |
| <code>/blast:impl</code>            | <code>&lt;nazwa&gt; [zadania] [-y]</code>                         | Implementacja w trybie TDD (persona Forge)                                        |
| <code>/blast:complete</code>        | <code>&lt;nazwa&gt;</code>                                        | Oznaczenie funkcjonalności jako wdrożonej, aktualizacja rejestru (persona Ledger) |
| <code>/blast:security</code>        | <code>&lt;nazwa&gt; [--fix]</code>                                | Audyt bezpieczeństwa z panelem JURY_3_FLASH3 (persona Sentinel)                   |
| <code>/blast:steering</code>        | <code> [--learn]</code>                                           | Rozruch lub synchronizacja warstwy kierującej (persona Cartographer)              |
| <code>/blast:steering-custom</code> | <code>&lt;temat&gt;</code>                                        | Generowanie pliku rozszerzającego pamięć projektu (persona Specialist)            |



## A.2. Fazy walidacji

| Komenda                | Argumenty                       | Opis                                                                           |
|------------------------|---------------------------------|--------------------------------------------------------------------------------|
| /blast:validate-gap    | <nazwa>                         | Analiza luki w stosunku do istniejącego kodu (persona Bridge)                  |
| /blast:validate-design | <nazwa> [--no-debate]           | Walidacja projektu z panelem JURY_3_FLASH3 (persona Crucible)                  |
| /blast:validate-tasks  | <nazwa> [--no-debate]           | Walidacja zadań pod kątem KISS oraz SOTA z panelem HYBRID (persona Pragmatist) |
| /blast:validate-impl   | <nazwa> [--prove] [--no-debate] | Walidacja implementacji z panelem HYBRID (persona Auditor)                     |

## A.3. Cykl życia

| Komenda          | Argumenty              | Opis                                                                                          |
|------------------|------------------------|-----------------------------------------------------------------------------------------------|
| /blast:approve   | <nazwa> <faza>         | Manualna akceptacja fazy specyfikacji                                                         |
| /blast:evolve    | <nazwa> "<zmiana>"     | Generowanie specyfikacji różnicowej dla wdrożonej funkcjonalności (persona Delta)             |
| /blast:deprecate | <nazwa> --reason "..." | Oznaczenie funkcjonalności jako wycofanej, generowanie przewodnika migracji (persona Curator) |
| /blast:status    | [<nazwa>]              | Wyświetlenie stanu jednej lub wszystkich specyfikacji                                         |

## A.4. Kompozycje i komendy meta

| Komenda     | Argumenty                                            | Opis                             |
|-------------|------------------------------------------------------|----------------------------------|
| /blast:full | "<opis>" [--auto] [--research] [--validate] [--push] | Pełen potok faz, wszystkie etapy |

| Komenda          | Argumenty                                              | Opis                                                                        |
|------------------|--------------------------------------------------------|-----------------------------------------------------------------------------|
| /blast:quick     | "<opis>" [--auto] [--research]                         | Potok ograniczony do specyfikacji (init, req, design, tasks)                |
| /blast:tiny      | "<opis>"                                               | Skompresowana specyfikacja dla trywialnych funkcjonalności (persona Sprint) |
| /blast:debate    | <nazwa> <temat> [--protocol A/B/C/D]                   | Uruchomienie debaty wielomodelowej na zadany temat                          |
| /blast:review    | <nazwa> [--debate]                                     | Przegląd kodu wobec zasad (persona Compass)                                 |
| /blast:learn     | [--lessons --calibrate --routing --refresh-sota --all] | Agregator pętli samodoskonalenia                                            |
| /blast:lint      | <pliki>                                                | Drugi przebieg analizy statycznej napędzany lokalnym modelem Qwen           |
| /blast:graph     | [<nazwa>]                                              | Wyciągnięcie grafu zależności ze specyfikacji                               |
| /blast:drift     | <nazwa>                                                | Wykrycie odchyleń pomiędzy specyfikacją a kodem (persona Tracker)           |
| /blast:telemetry | [--summary --detail]                                   | Wyświetlenie telemetrii (częstotliwość debat, koszty, opóźnienia)           |
| /blast:ping-llm  | brak                                                   | Test sprawdzający funkcjonowanie mostu MCP do modeli                        |
| /blast:push      | [<nazwa>]                                              | Utworzenie commita oraz wysłanie zmian do zdalnego repozytorium             |
| /blast:help      | [<komenda>]                                            | Wyświetlenie dokumentacji komend                                            |

## Dodatek B. Słownik

**Artykuł:** jeden z jedenastu wiążących zapisów governance w pliku .blast/CONSTITUTION.md. Artykuły pozostają stabilne, warstwa kierująca operacyjna może ulegać zmianom.

**Bramka akceptacji:** deterministyczna kontrola pomiędzy fazami specyfikacji egzekwowana przez mechanizm kontrolny na poziomie środowiska uruchomieniowego (mechanizm blast-approval-gate.py).

**Debata:** wielojurorska ocena tematu (poprawność projektu, zgodność implementacji, postawa bezpieczeństwa, dopasowanie do KISS oraz SOTA). Uruchamiana komendą /blast:debate bezpośrednio lub jako wywołanie zagnieżdżone z gałęzi FIRE komendy walidacji.

**EARS:** Easy Approach to Requirements Syntax (uproszczone podejście do składni wymagań). Domyślny format wymagań w blast, ujęty w schemat: gdy zachodzi <warunek>, system <nazwa> ma <reakcja>, oraz jego warianty.

**Faza:** jeden z etapów potoku pracy w blast (inicjalizacja, specyfikacja wymagań, faza badawcza, projekt techniczny, walidacja projektu, rozpisanie zadań, walidacja zadań, implementacja, walidacja implementacji, zamknięcie, audyt bezpieczeństwa, synchronizacja pamięci, opcjonalnie: wysłanie do repozytorium).

**FIRE oraz SKIP:** decyzje kierujące w komendach walidacji, czy faza walidacji uruchamia kompozycję debaty (FIRE), czy działa w trybie samodzielny (SKIP). Obliczane deterministycznie przez komendę walidacji, nie przez agenta.

**HYBRID:** panel dwuosobowy: Sonnet z Qwen, agregator Haiku. Stosowany dla walidacji implementacji oraz walidacji zadań.

**JURY\_3\_FLASH3:** panel trzyosobowy: Opus, Qwen oraz Gemini Flash, agregator Haiku. Stosowany dla walidacji projektu, audytu bezpieczeństwa oraz wysokokrytycznego przeglądu kodu.

**MCP:** Model Context Protocol (protokół kontekstu modelu). Używany przez blast do udostępnienia lokalnych modeli z biblioteki Ollama oraz API Gemini jako narzędzi środowiska Claude Code, za pośrednictwem mostu zaimplementowanego w pliku .claude/mcp/blast-llm-bridge.py.

**Persona:** nazwana tożsamość (Atlas, Forge, Loom, ...) noszona przez jedną lub więcej definicji agentów, ze sformułowaną rolą, stylem oraz mechanizmem wykrywania własnej stroniczości.

**Tryb prywatności:** ustawienie `spec.json.privacy` na wartość `local-only` (tylko lokalnie). Blokuje każde wywołanie zewnętrznego modelu poprzez mechanizm `blast-privacy-gate.py` i kieruje pracę na lokalne kompozycje paneli.

**Tryb dowodu:** komenda `/blast:validate-impl --prove`. Uruchamia blok strategii weryfikacji z pliku `design.md` jako sondy wykonawcze (pojedynczy test, kontrola pierwszego startu, sonda end-to-end) i raportuje status każdej.

**Rola zastępcza:** rola jurora odgrywana przez agenta zamiast prawdziwego podagenta lub narzędzia MCP. Zakazana przez kluczowe ograniczenia agregatora; zapisywana w polu `JUROR_DEGRADATIONS`, gdy juror jest faktycznie niedostępny.

**Warstwa kierująca:** katalog `.blast/steering`. Pamięć operacyjna projektu o długim cyklu życia: produkt, technologia, struktura, rejestr komponentów (INVENTORY), kierowanie zapytań do modeli, polityka kosztowa.

**Kierowanie zapytań do tańszego modelu:** praktyka kierowania każdej fazy do najtańszego modelu zdolnego dostarczyć wymaganą jakość (Haiku do szablonowania, Sonnet do wnioskowania o kodzie, Opus do architektury, Qwen do lokalnej generacji kodu, Gemini do różnorodności dostawców w panelach).